

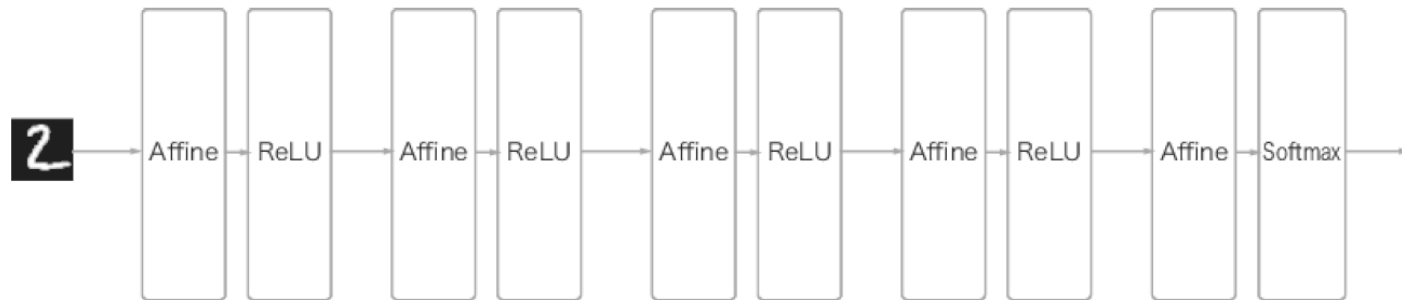
Convolution Neural Network

Prof. Tae-Hyoung Park

Dept. of Intelligent Systems & Robotics, CBNU

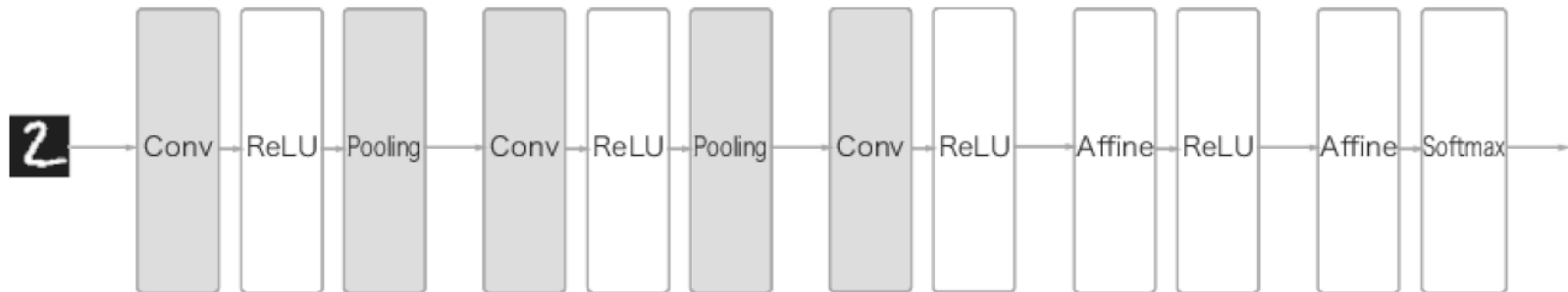
CNN Structure

- Deep neural network (fully connected network)



영상 데이터 -> 1차원 벡터 : 입력 영상의 '형상' 무시
(ex) 32 pixel x 32 pixel x 3 channel image -> 3072 x 1 vector

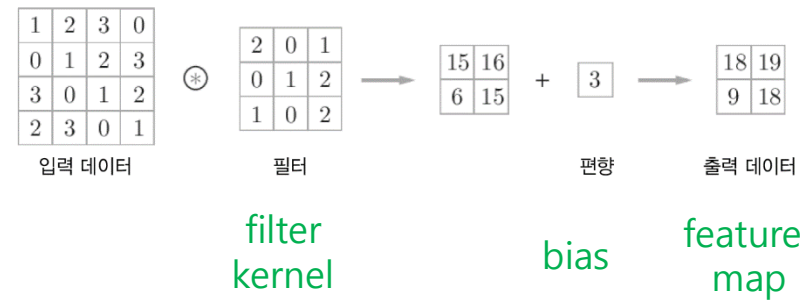
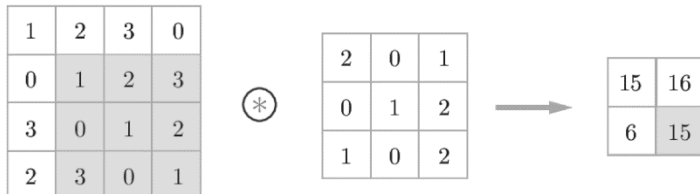
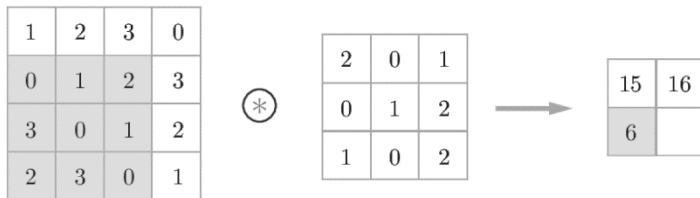
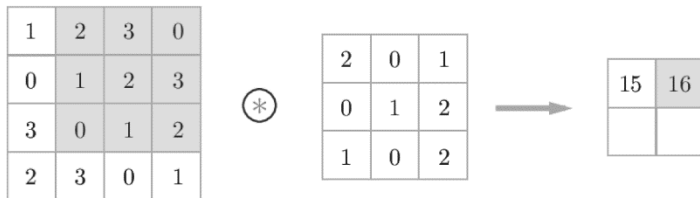
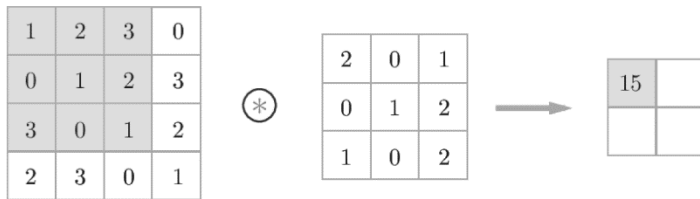
- Convolution neural network



영상 데이터 -> 3차원 벡터 (가로,세로,채널) : 입력 영상의 '형상' 유지

Convolution Layer

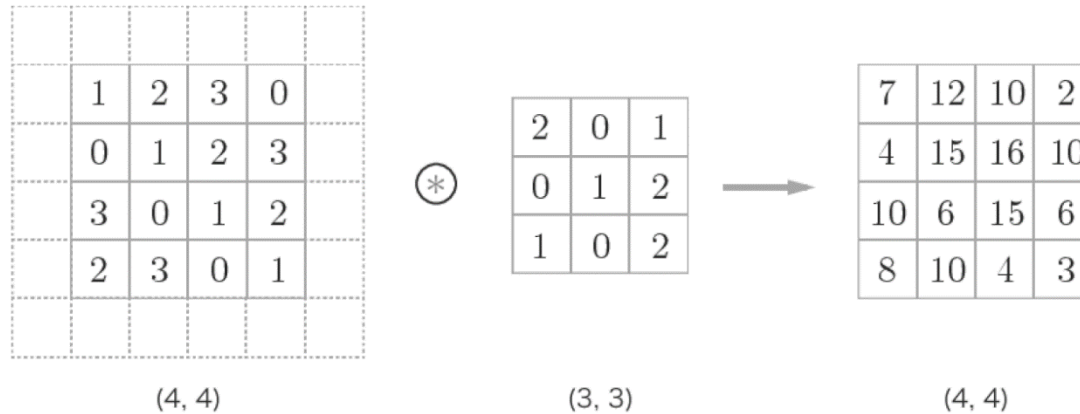
- Convolution 연산
 - 이웃 pixel 들의 공간적 특성값 (spatial feature)



Convolution Layer

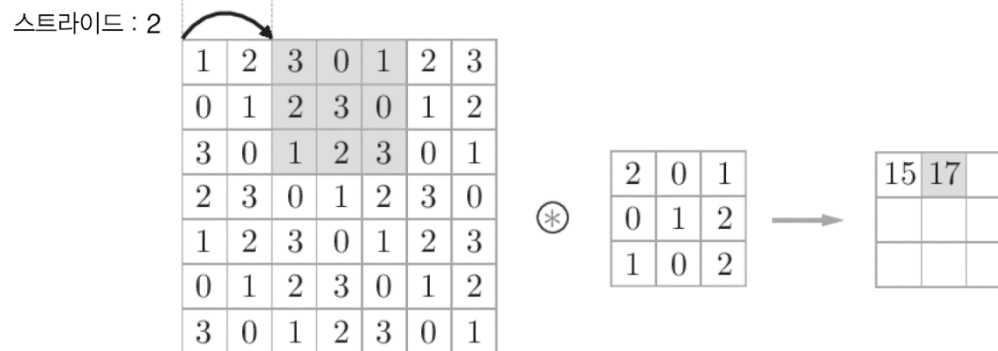
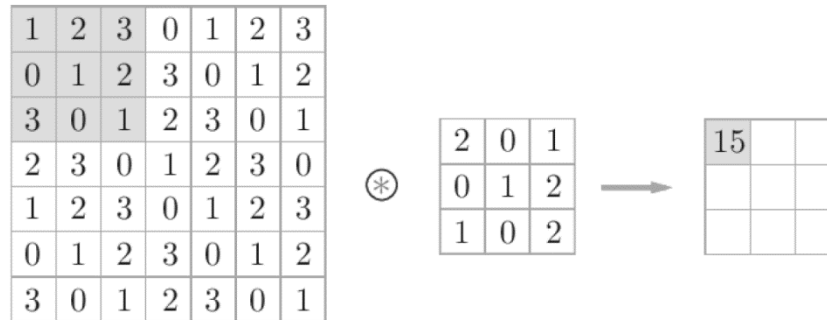
- Padding

- 영상 모서리의 convolution 연산을 위하여, 입력 데이터 주변을 특정값으로 채움



Convolution Layer

- Stride
 - Filter 를 적용하는 위치 간격



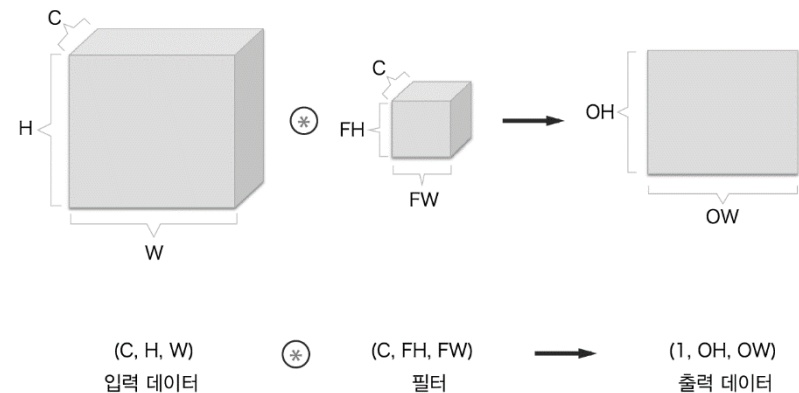
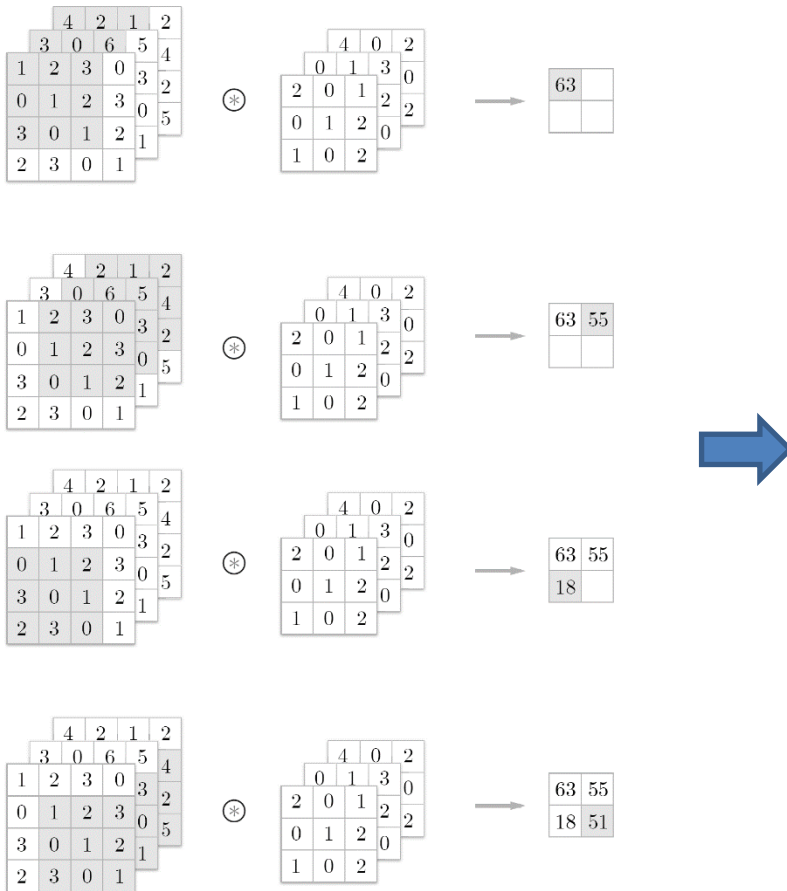
$$OH = \frac{H + 2P - FH}{S} + 1$$

$$OW = \frac{W + 2P - FW}{S} + 1$$

입력크기: (H,W), Filter 크기: (FH, FW), 출력크기: (OH, OW)
 Padding: P, Stride: S

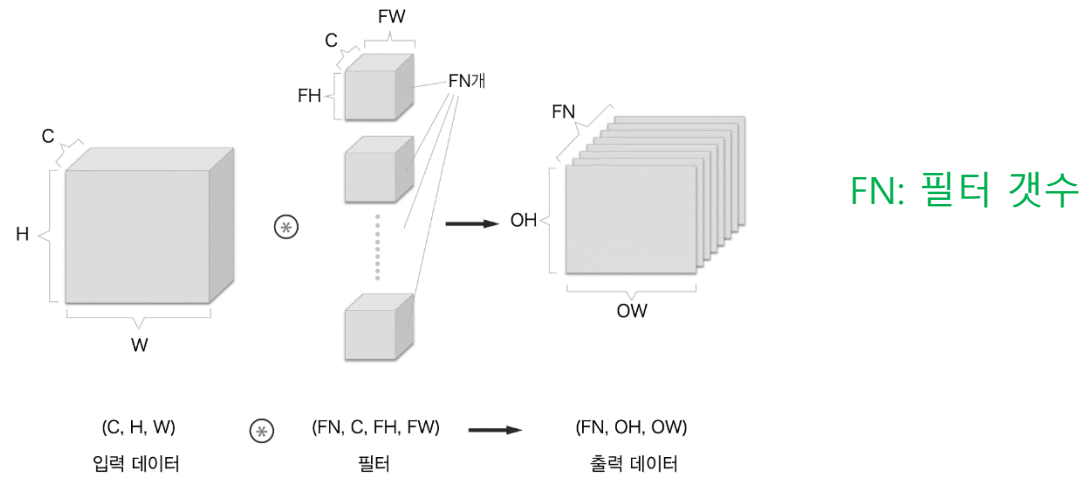
Convolution Layer

- Color image convolution
 - Convolution for 3 channels (R,G,B)
 - 3D data convolution

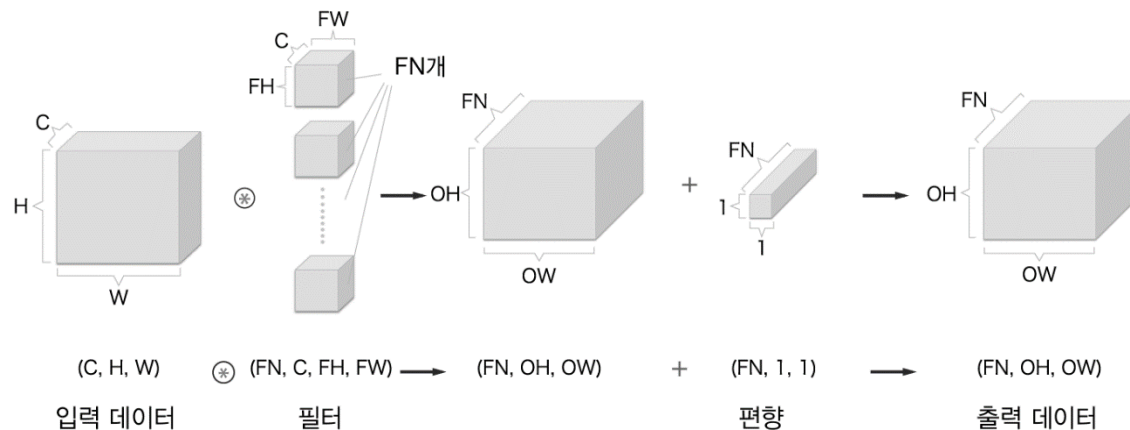


Convolution Layer

- 여러 개의 filter 사용



- Bias 추가



Convolution Layer

- Hyperparameters

- Number of filters **K**
- The filter size **F**
- The stride **S**
- The zero padding **P**

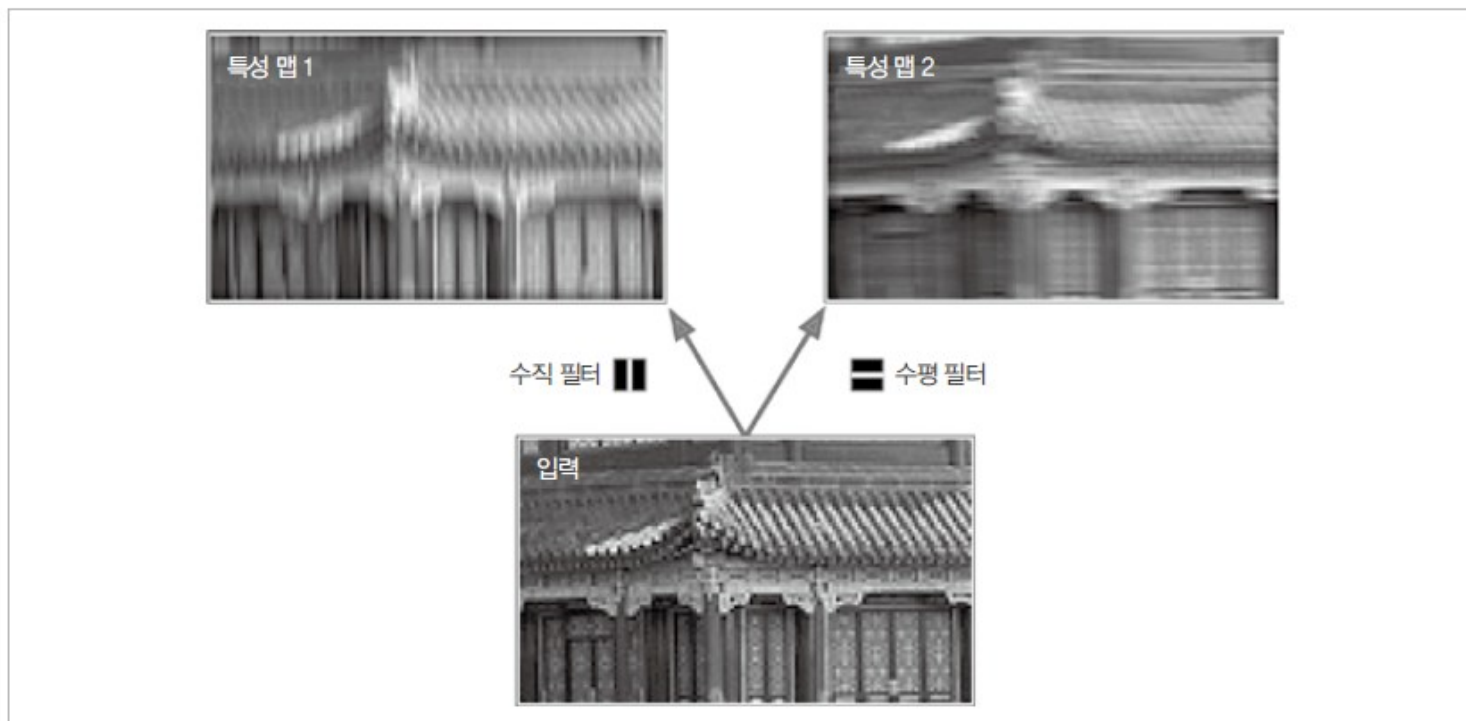
- 학습 parameter

F^2CK and K biases

(C: number of channels)

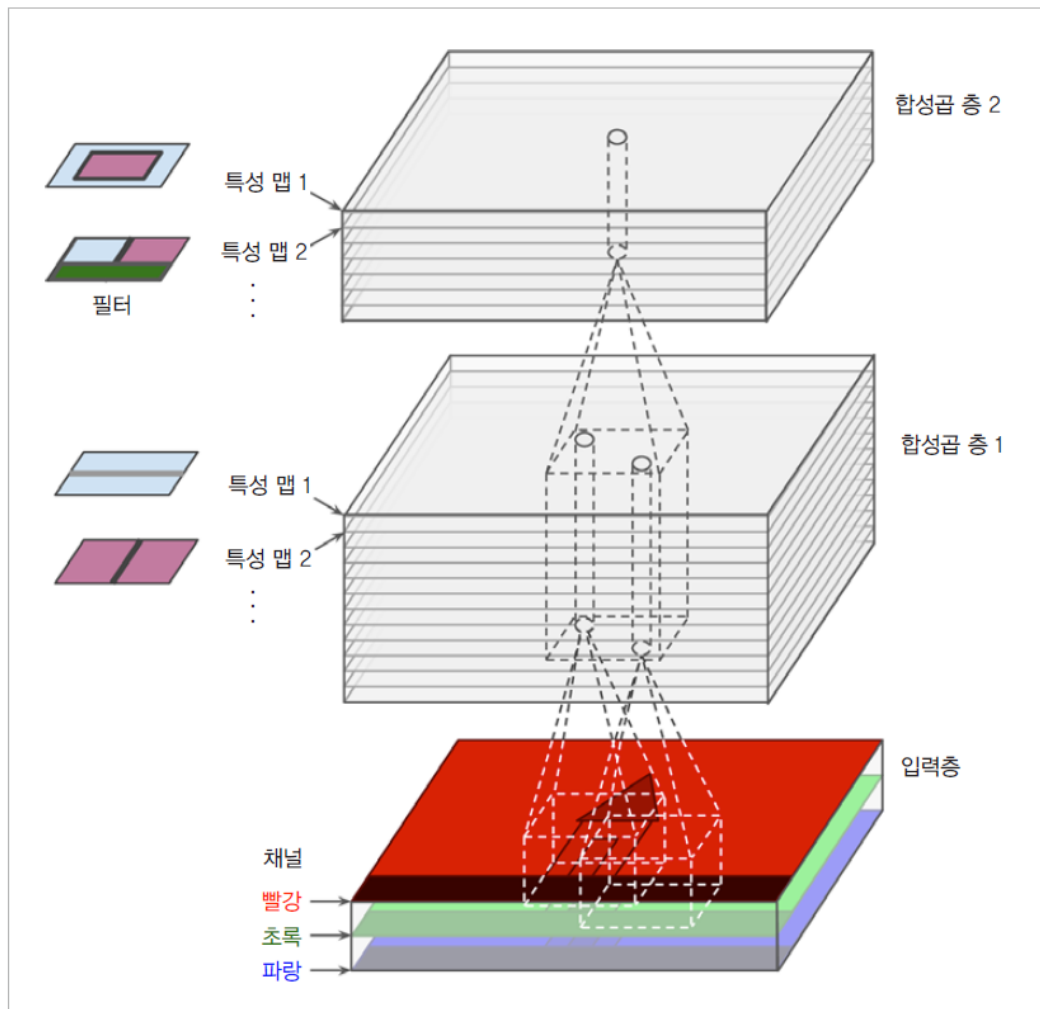
Convolution Layer

- Filter (kernel) & feature map



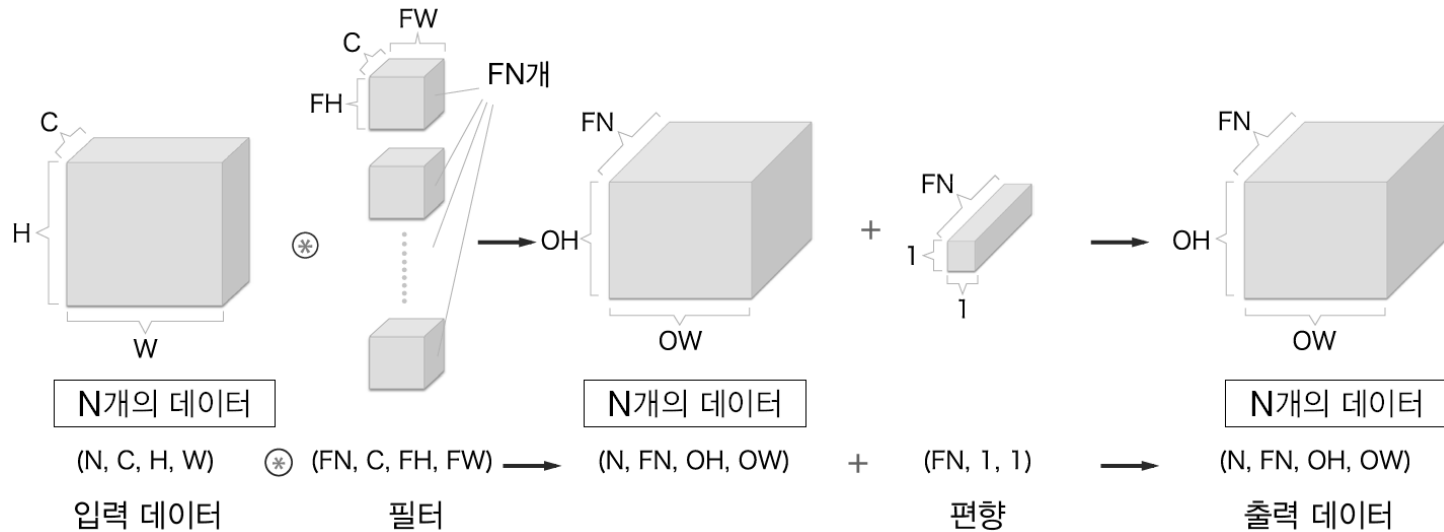
Convolution Layer

- Filter & Feature map



Convolution Layer

- Batch 처리



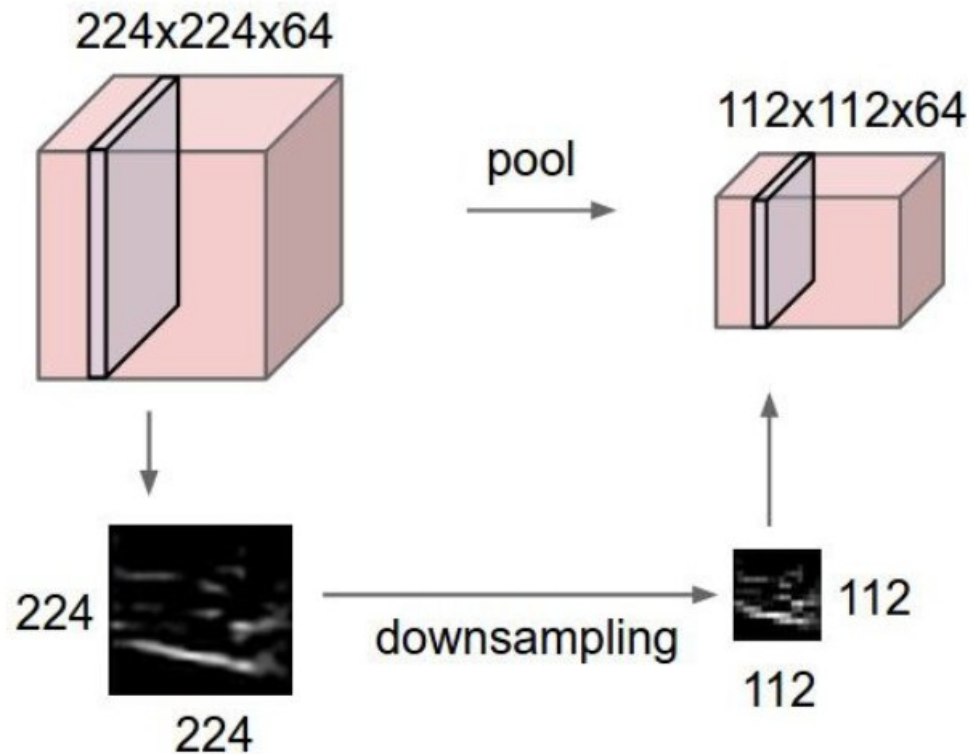
N: batch 데이터 수

- 4차원 데이터 (Tensor) 의 흐름
- 연산내용: 행렬 곱셈 & 덧셈
- Convolution 연산으로 local feature 를 추출하여 feature map 출력

Pooling Layer

- 목적

- 계산량과 메모리 사용량 감소
- 학습 파라미터 수 감소
- ☞ 과적합 문제 개선



Pooling Layer

- Pooling layer
 - 가로 세로 방향의 공간을 줄이는 연산
 - 종류: Max pooling, average pooling 등
 - 학습 대상이 되는 parameter 가 없음

pooling: 취합/선별

Max-pooling

1	2	1	0
0	1	2	3
3	0	1	2
2	4	0	1



2	

1	2	1	0
0	1	2	3
3	0	1	2
2	4	0	1



2	3

1	2	1	0
0	1	2	3
3	0	1	2
2	4	0	1



2	3
4	

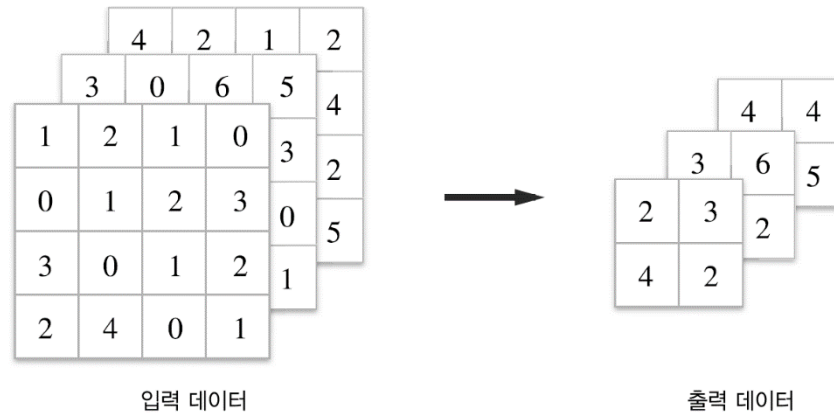
1	2	1	0
0	1	2	3
3	0	1	2
2	4	0	1



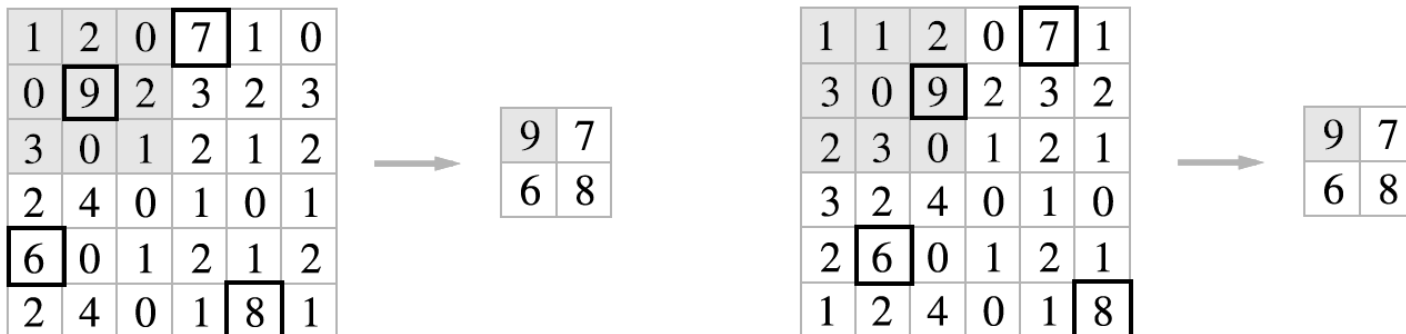
2	3
4	2

Pooling Layer

- 채널 수 유지



- 입력의 변화에 강인

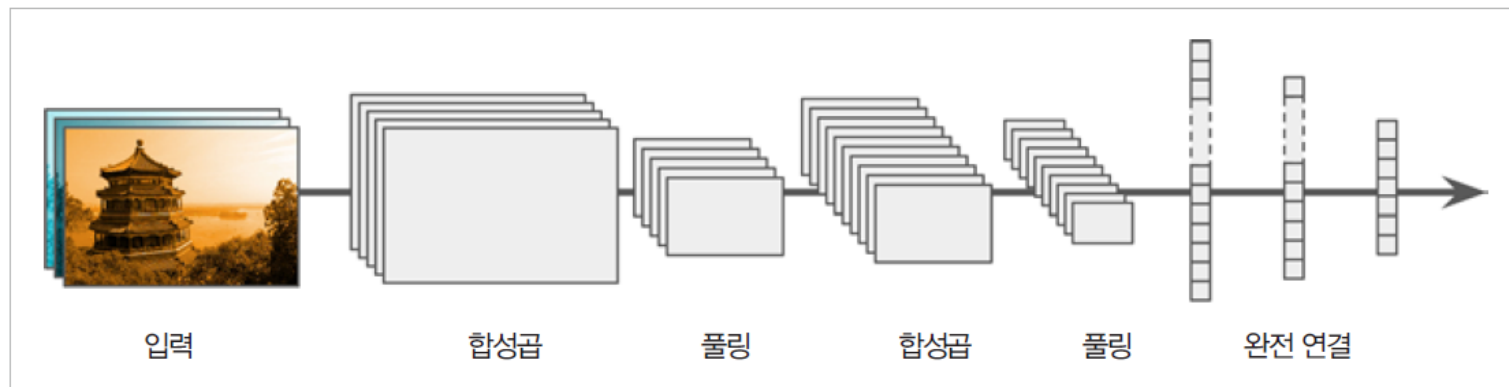


Pooling Layer

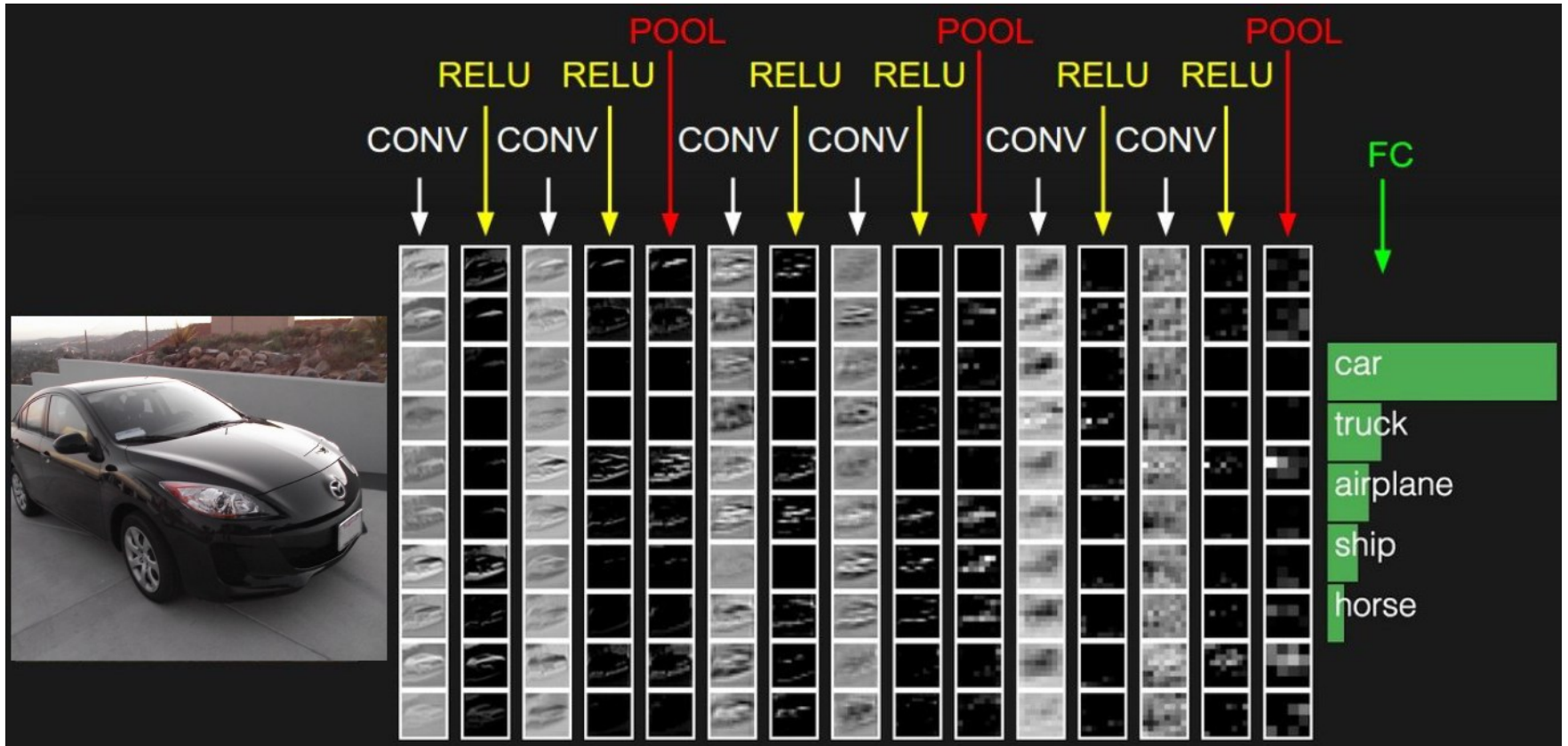
- Hyperparameter
 - Spatial extent **F**
 - Stride **S**
- 학습 parameter
 - none

기본 CNN 구조

- Typical CNN
 - Convolution layer + Pooling layer + Fully connected layer



기본 CNN 구조



CNN 구조

- AlexNet

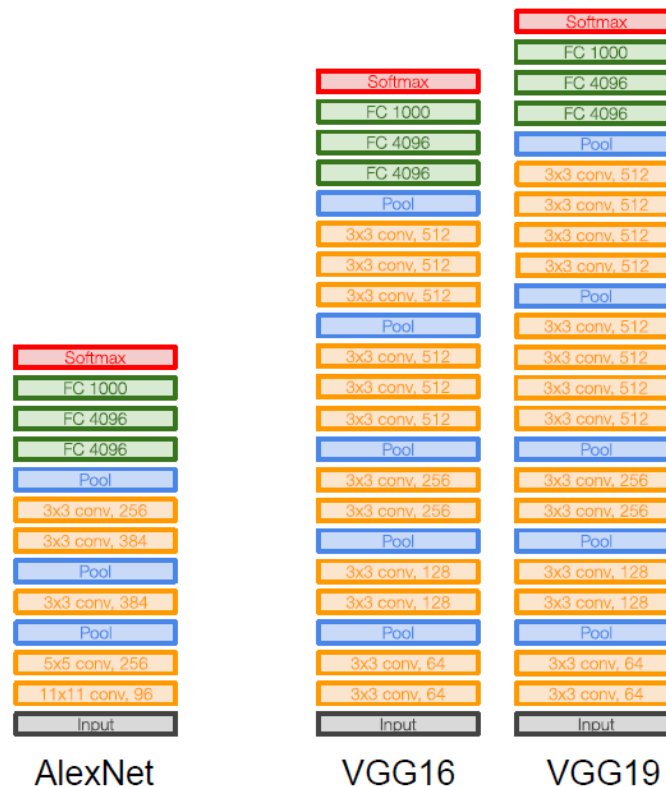
- Alex Krizhevsky (2012)
- ImageNet 대회 우승 (error rate: 17%)
- 특징
 - 연속적 Conv. Layer
 - DropOut 적용:50%
 - Data augmentation 기법 적용

층	종류	특성 맵	크기	커널 크기	스트라이드	패딩	활성화 함수
출력	완전 연결	-	1,000	-	-	-	Softmax
F10	완전 연결	-	4,096	-	-	-	ReLU
F9	완전 연결	-	4,096	-	-	-	ReLU
F8	최대 풀링	256	6×6	3×3	2	valid	-
C7	합성곱	256	13×13	3×3	1	same	ReLU
C6	합성곱	384	13×13	3×3	1	same	ReLU
C5	합성곱	384	13×13	3×3	1	same	ReLU
S4	최대 풀링	256	13×13	3×3	2	valid	-
C3	합성곱	256	27×27	5×5	1	same	ReLU
S2	최대 풀링	96	27×27	3×3	2	valid	-
C1	합성곱	96	55×55	11×11	4	valid	ReLU
입력	입력	3 (RGB)	227×227	-	-	-	-

CNN 구조

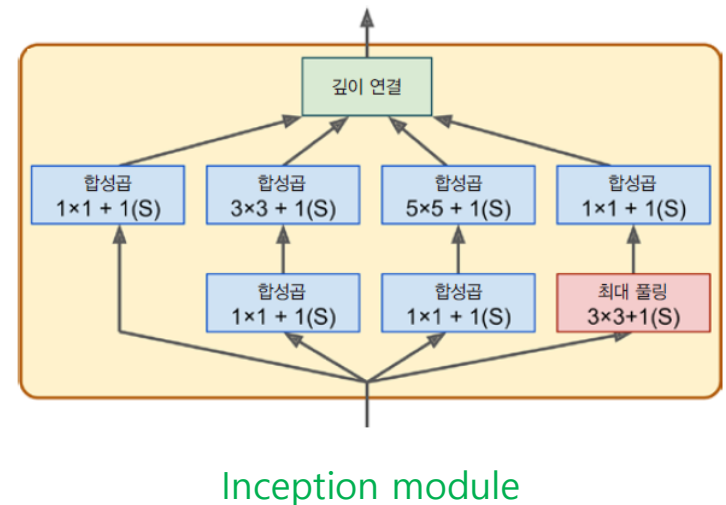
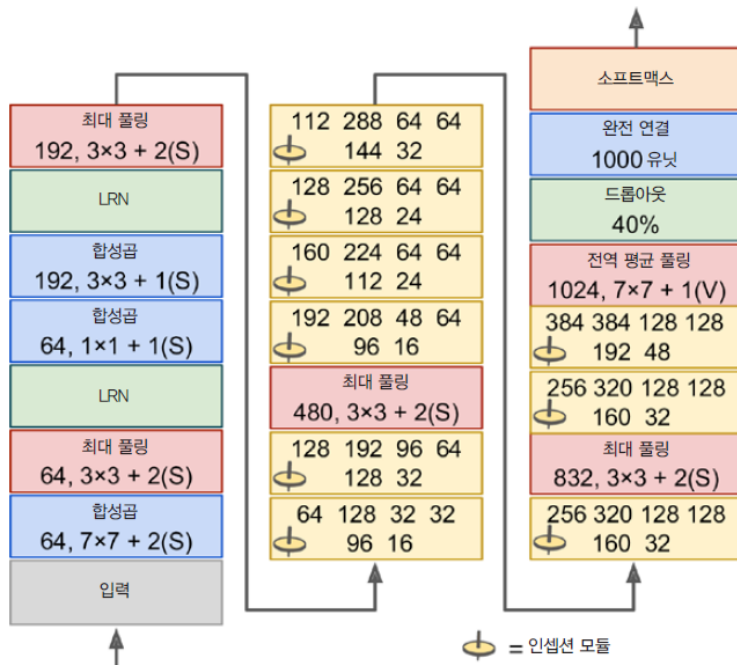
- VGGNet

- Simonyan & Zisserman, 2014
- Alex net 대비 Conv. Kernel size 를 줄이고 layer 를 늘림
 - 여러 개의 3x3 conv. 는 1개의 7x7 conv. 와 effective receptive field 가 같음
 - Deeper, more non-linearities



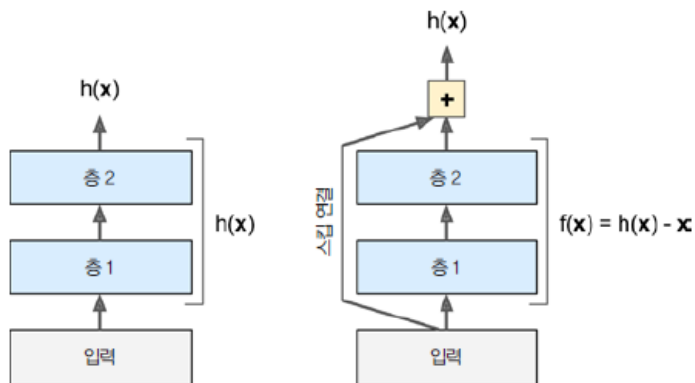
CNN 구조

- GoogleLeNet
 - Christian Szegedy (2015)
 - ILSVRC 2014 대회 우승 (error rate: 7%)
 - 특징
 - Inception module
 - 22 layers
 - 파라미터 수 감소 (AlexNet 대비 12배, VGG-16 대비 27배 감소)

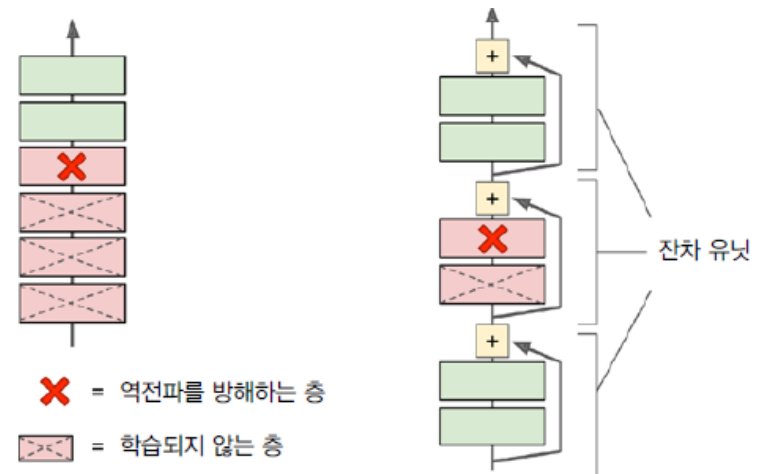


CNN 구조

- ResNet (residual network)
 - Kaiming He (2015)
 - ILSVRC 2015 대회 우승 (error rate 3.6%)
 - 특징
 - 152 layers
 - Skip connection & residual learning: back propagation 개선



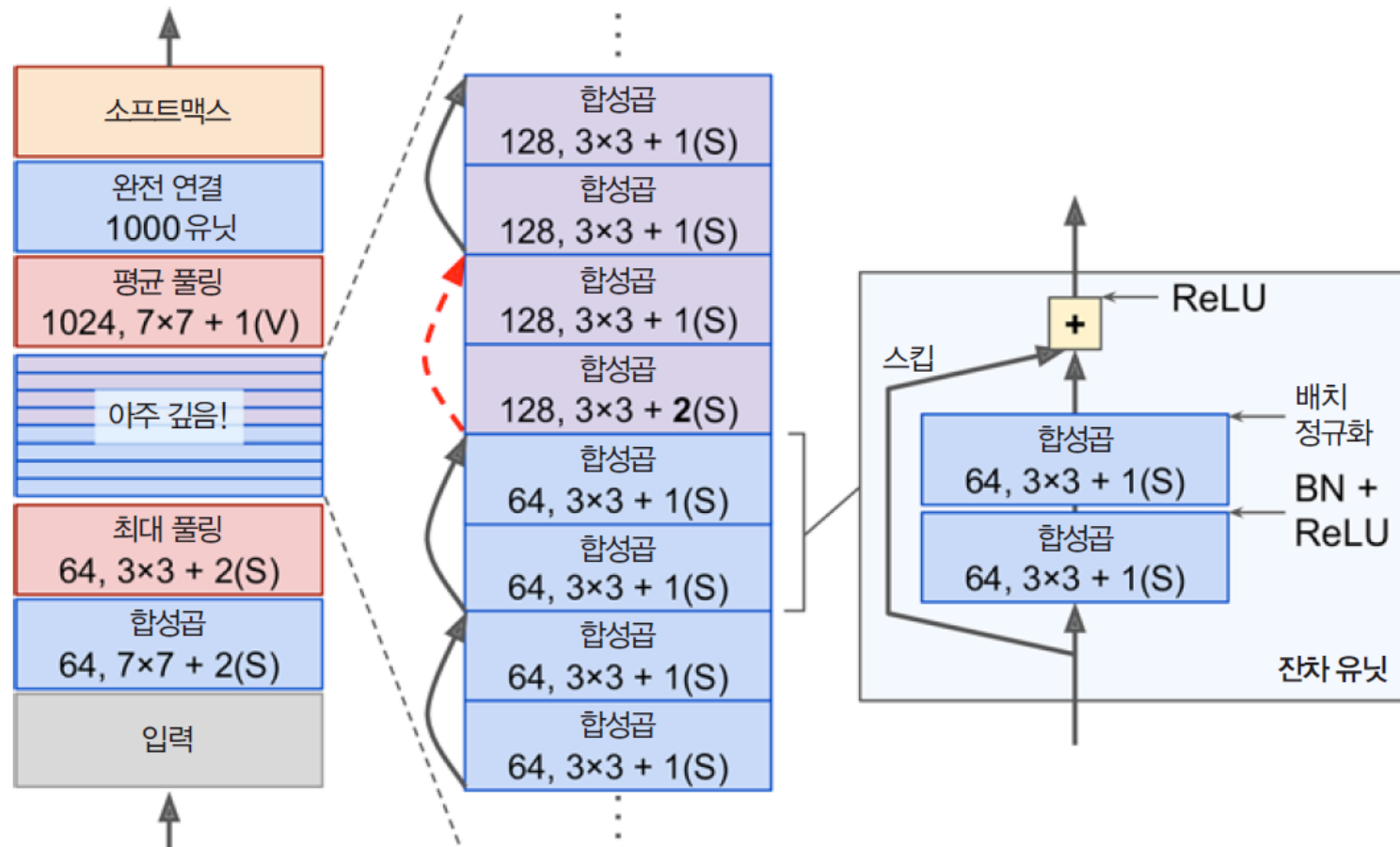
skip connection



Residual learning

CNN 구조

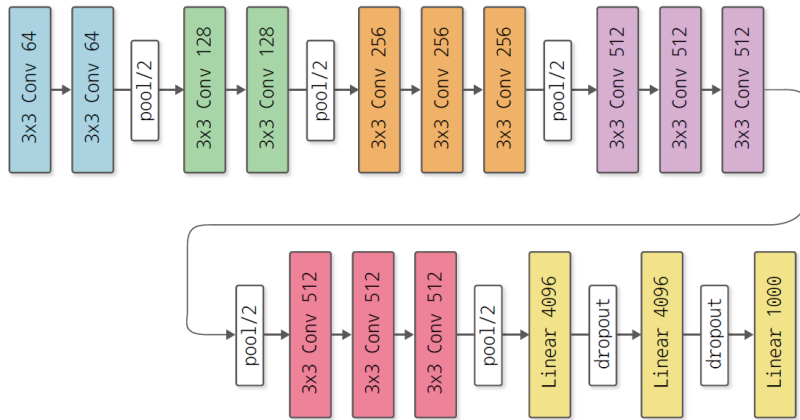
- ResNet (residual network)



VGG16 구현

DeZeroWmodels.py

- Model



```
class VGG16(Model):
    WEIGHTS_PATH = 'https://github.com/koki0702/dezero-models/releases/download/v0.1/vgg16.npz'

    def __init__(self, pretrained=False):
        super().__init__()
        self.conv1_1 = L.Conv2d(64, kernel_size=3, stride=1, pad=1)
        self.conv1_2 = L.Conv2d(64, kernel_size=3, stride=1, pad=1)
        self.conv2_1 = L.Conv2d(128, kernel_size=3, stride=1, pad=1)
        self.conv2_2 = L.Conv2d(128, kernel_size=3, stride=1, pad=1)
        self.conv3_1 = L.Conv2d(256, kernel_size=3, stride=1, pad=1)
        self.conv3_2 = L.Conv2d(256, kernel_size=3, stride=1, pad=1)
        self.conv3_3 = L.Conv2d(256, kernel_size=3, stride=1, pad=1)
        self.conv4_1 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.conv4_2 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.conv4_3 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.conv5_1 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.conv5_2 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.conv5_3 = L.Conv2d(512, kernel_size=3, stride=1, pad=1)
        self.fc6 = L.Linear(4096)
        self.fc7 = L.Linear(4096)
        self.fc8 = L.Linear(1000)

        if pretrained:
            weights_path = utils.get_file(VGG16.WEIGHTS_PATH)
            self.load_weights(weights_path)

    def forward(self, x):
        x = F.relu(self.conv1_1(x))
        x = F.relu(self.conv1_2(x))
        x = F.pooling(x, 2, 2)
        x = F.relu(self.conv2_1(x))
        x = F.relu(self.conv2_2(x))
        x = F.pooling(x, 2, 2)
        x = F.relu(self.conv3_1(x))
        x = F.relu(self.conv3_2(x))
        x = F.relu(self.conv3_3(x))
        x = F.pooling(x, 2, 2)
        x = F.relu(self.conv4_1(x))
        x = F.relu(self.conv4_2(x))
        x = F.relu(self.conv4_3(x))
        x = F.pooling(x, 2, 2)
        x = F.relu(self.conv5_1(x))
        x = F.relu(self.conv5_2(x))
        x = F.relu(self.conv5_3(x))
        x = F.pooling(x, 2, 2)
        x = F.reshape(x, (x.shape[0], -1))
        x = F.dropout(F.relu(self.fc6(x)))
        x = F.dropout(F.relu(self.fc7(x)))
        x = self.fc8(x)
        return x
```

VGG16 구현

- Pre-trained weight 사용

```
import numpy as np
from PIL import Image
import dezero
from dezero.models import VGG16

url = 'https://github.com/oreilly-japan/deep-learning-from-scratch-3/raw/images/zebra.jpg'
img_path = dezero.utils.get_file(url)
img = Image.open(img_path)

x = VGG16.preprocess(img)
x = x[np.newaxis]

model = VGG16(pretrained=True)
with dezero.test_mode():
    y = model(x)
predict_id = np.argmax(y.data)

labels = dezero.datasets.ImageNet.labels()
print(labels[predict_id])
```



zebra